

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE CODE: CSA1401

COURSE NAME: COMPILER DESIGN

COURSE OUTCOME COVERED

CO3: *Analyze syntax-directed translation method using synthesized and inherited attributes to generate a Parser Tree. (BL4)*

Assessment Tool Weightage: 40%

QUESTIONS:5

Duration :60 Mins
On Class
Total Marks:50

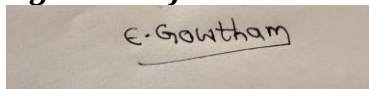
Annexure A

EXPERIMENTS

Code of Conduct:

I Enamala Gowtham (192311190) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

A photograph of a piece of paper with the handwritten signature "E. Gowtham" in black ink. The signature is written in a cursive style and is underlined.

Annexure A

EXPERIMENTS

Q1. Calculator Using YACC

Problem

Evaluate:

$3 + 4 * 5$

Expected result:

23

Because multiplication has higher precedence than addition.

Step 1: Create q1.l

```
%{
#include "q1.tab.h"
#include <stdlib.h>
%}

%%

[0-9]+    {
    yylval = atoi(yytext);
    return NUMBER;
}

[ \t]+    ;

\n        return '\n';

"+"       return '+';
"*"       return '*';

.         return yytext[0];

%%

int yywrap()
{
    return 1;
}
```

Step 2: Create q1.y

```
%{
#include <stdio.h>
#include <stdlib.h>

int yylex();
void yyerror(const char *s);
%}

%token NUMBER

%left '+'
```

```

%left '*'

%%

input:
    expr '\n'
    {
        printf("Computed Result = %d\n", $1);
    }
    ;

expr:
    expr '+' expr
    {
        $$ = $1 + $3;
    }
    | expr '*' expr
    {
        $$ = $1 * $3;
    }
    | NUMBER
    {
        $$ = $1;
    }
    ;

%%

void yyerror(const char *s)
{
    printf("Syntax Error: %s\n", s);
}

int main()
{
    printf("Enter arithmetic expression: ");
    yyparse();

    return 0;
}

```

Step 3: Compile

Windows

flex q1.l

bison -d q1.y

gcc lex.yy.c q1.tab.c -o q1.exe

q1.exe

Step 4: Input

3 + 4 * 5

Output

Enter arithmetic expression: 3 + 4 * 5

Computed Result = 23

Step 5: Semantic Action Explanation

For:

$3 + 4 * 5$

First:

$4 * 5 = 20$

Then:

$3 + 20 = 23$

The semantic actions are:

$$$ = \$1 + \$3;$

and:

$$$ = \$1 * \$3;$

Meaning

Symbol Meaning

$\$1$ First RHS value

$\$2$ Second RHS value

$\$3$ Third RHS value

$$$$ LHS value

Result

Computed Result = 23

Q2. Construct an AST Using YACC

Problem

Construct an Abstract Syntax Tree for:

$a + b * c$

The AST should be:

```
      +
     /\
    a  *
     /\
    b  c
```

Step 1: Create q2.l

```
%{
```

```
#include "q2.tab.h"
```

```
#include <string.h>
```

```
#include <stdlib.h>
```

```
%}
```

```
%%
```

```
[a-zA-Z][a-zA-Z0-9]* {
    yylval.str = strdup(yytext);
    return ID;
}
```

```
[ \t]+ ;
```

```
\n      return '\n';
```

```
"+"      return '+';
```

```
"*"      return '*';
```

```
.        return yytext[0];
```

```
%%
```

```
int yywrap()  
{  
    return 1;  
}
```

Step 2: Create q2.y

```
%{  
#include <stdio.h>  
#include <stdlib.h>  
#include <string.h>  
  
int yylex();  
void yyerror(const char *s);
```

```
typedef struct Node  
{  
    char value[20];  
    struct Node *left;  
    struct Node *right;  
} Node;
```

```
Node* createNode(char *value, Node *left, Node *right);  
void preorder(Node *root);  
void inorder(Node *root);  
void postorder(Node *root);  
void printTree(Node *root, int level);  
%}
```

```
%union  
{  
    char *str;  
    Node *node;  
}
```

```
%token <str> ID
```

```
%type <node> expr
```

```
%left '+'  
%left '**'
```

```
%%
```

```
input:  
    expr '\n'  
    {  
        printf("\nAST:\n");  
        printTree($1, 0);  
  
        printf("\nPreorder: ");  
        preorder($1);  
  
        printf("\nInorder: ");
```

```

        inorder($1);

        printf("\nPostorder: ");
        postorder($1);

        printf("\n");
    }
;

expr:
    expr '+' expr
    {
        $$ = createNode("+", $1, $3);
    }
    | expr '*' expr
    {
        $$ = createNode("*", $1, $3);
    }
    | ID
    {
        $$ = createNode($1, NULL, NULL);
    }
;

%%

Node* createNode(char *value, Node *left, Node *right)
{
    Node *newNode = (Node*)malloc(sizeof(Node));

    strcpy(newNode->value, value);
    newNode->left = left;
    newNode->right = right;

    return newNode;
}

void preorder(Node *root)
{
    if(root == NULL)
        return;

    printf("%s ", root->value);
    preorder(root->left);
    preorder(root->right);
}

void inorder(Node *root)
{
    if(root == NULL)
        return;

    inorder(root->left);
    printf("%s ", root->value);
    inorder(root->right);
}

```

```

}

void postorder(Node *root)
{
    if(root == NULL)
        return;

    postorder(root->left);
    postorder(root->right);
    printf("%s ", root->value);
}

void printTree(Node *root, int level)
{
    int i;

    if(root == NULL)
        return;

    for(i = 0; i < level; i++)
        printf(" ");

    printf("%s\n", root->value);

    printTree(root->left, level + 1);
    printTree(root->right, level + 1);
}

void yyerror(const char *s)
{
    printf("Syntax Error: %s\n", s);
}

int main()
{
    printf("Enter expression: ");
    yyparse();

    return 0;
}

```

Step 3: Compile

flex q2.l

bison -d q2.y

gcc lex.yy.c q2.tab.c -o q2.exe

q2.exe

Step 4: Input

a + b * c

Step 5: Output

AST:

+
a
*
b
c

Preorder: + a * b c
Inorder: a + b * c
Postorder: a b c * +

Step 6: Tree
Draw this in your assessment:

```
      +
     /\
    a  *
     /\
    b  c
```

Traversals

Preorder:

+ a * b c

Inorder:

a + b * c

Postorder:

a b c * +

Result

The AST correctly represents the operator precedence:

b * c

is evaluated before:

a + ...

Q3. Syntax-Directed Definition Using Synthesized Attributes

Problem

Evaluate:

2 * 3 + 4

Expected:

10

Step 1: Create q3.l

```
%{
#include "q3.tab.h"
#include <stdlib.h>
%}

%%

[0-9]+ {
    yylval = atoi(yytext);
    return NUMBER;
}

[ \t]+ ;

\n    return '\n';

"+"   return '+';
```

```

    return '**';

    return yytext[0];

```

```

%%

```

```

int yywrap()
{
    return 1;
}

```

Step 2: Create q3.y

```

%{
#include <stdio.h>
#include <stdlib.h>

int yylex();
void yyerror(const char *s);
%}

```

```

%token NUMBER

```

```

%left '+'
%left '**'

```

```

%%

```

```

input:
    expr '\n'
    {
        printf("\nFinal Value = %d\n", $1);
    }
    ;

```

```

expr:
    expr '+' expr
    {
        $$ = $1 + $3;

        printf("Reduce: expr + expr => %d + %d = %d\n",
            $1, $3, $$);
    }
    | expr '**' expr
    {
        $$ = $1 * $3;

        printf("Reduce: expr * expr => %d * %d = %d\n",
            $1, $3, $$);
    }
    | NUMBER
    {
        $$ = $1;

        printf("Reduce: NUMBER => %d\n", $$);
    }

```

```

;

%%

void yyerror(const char *s)
{
    printf("Syntax Error: %s\n", s);
}

int main()
{
    printf("Enter expression: ");
    yyparse();

    return 0;
}

```

Step 3: Compile

```

flex q3.l
bison -d q3.y
gcc lex.yy.c q3.tab.c -o q3.exe
q3.exe

```

Step 4: Input

2 * 3 + 4

Step 5: Output

Enter expression: 2 * 3 + 4

```

Reduce: NUMBER => 2
Reduce: NUMBER => 3
Reduce: expr * expr => 2 * 3 = 6
Reduce: NUMBER => 4
Reduce: expr + expr => 6 + 4 = 10

```

Final Value = 10

Step 6: Attribute Propagation

For:

2 * 3 + 4

First

NUMBER → 2

Therefore:

\$\$ = \$1 = 2

Second

NUMBER → 3

Therefore:

\$\$ = \$1 = 3

Multiplication

expr * expr

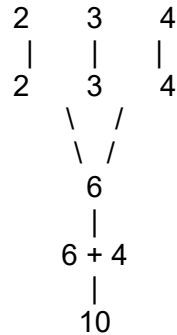
\$\$ = \$1 * \$3

Therefore:

\$\$ = 2 * 3

$$$ = 6$
 Addition
 $\text{expr} + \text{expr}$
 $$$ = \$1 + \$3$
 Therefore:
 $$$ = 6 + 4$
 $$$ = 10$

Bottom-Up Evaluation



Key point
 This is a synthesized attribute because information flows:
 Children $\rightarrow$ Parent

Q4. Name Equivalence and Structural Equivalence

Given:

type A = int;

type B = int;

A x;

B y;

We need to verify:

Name Equivalence

$A \neq B$

Structural Equivalence

A = int

B = int

Therefore $A \equiv B$ structurally.

Step 1: Create q4.l

```
%{
```

```
#include "q4.tab.h"
```

```
#include <string.h>
```

```
#include <stdlib.h>
```

```
%}
```

```
%%
```

```
"type"    return TYPE;
```

```
"int"     return INT;
```

```
"float"   return FLOAT;
```

```
[A-Za-z_][A-Za-z0-9_]* {
```

```
    yylval.str = strdup(yytext);
```

```
    return ID;
```

```

    }

    "="      return '=';
    ",'"     return ',';

```

```

[ \t\n]+    ;

. return yytext[0];

%%

```

```

int yywrap()
{
    return 1;
}

```

Step 2: Create q4.y

```

%{
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

```

```

int yylex();
void yyerror(const char *s);

```

```

#define MAX 100

```

```

typedef struct
{
    char name[30];
    char type[30];
    int isAlias;
} Symbol;

```

```

Symbol table[MAX];
int count = 0;

```

```

void addType(char *name, char *type);
void addVariable(char *name, char *type);
char* lookupType(char *name);
int nameEquivalent(char *a, char *b);
int structuralEquivalent(char *a, char *b);
%}

```

```

%union
{
    char *str;
}

```

```

%token TYPE INT FLOAT
%token <str> ID

```

```

%%

```

```

program:

```

```

    declarations
    ;

declarations:
    declarations declaration
    | declaration
    ;

declaration:
    TYPE ID '=' INT ';'
    {
        addType($2, "int");
    }
    | TYPE ID '=' FLOAT ';'
    {
        addType($2, "float");
    }
    | ID ID ';'
    {
        char *baseType = lookupType($1);

        if(baseType != NULL)
        {
            addVariable($2, baseType);

            printf("Variable %s declared as type %s\n",
                $2, $1);
        }
        else
        {
            printf("Error: Unknown type %s\n", $1);
        }
    }
    ;

%%

```

```

void addType(char *name, char *type)
{
    strcpy(table[count].name, name);
    strcpy(table[count].type, type);
    table[count].isAlias = 1;

    printf("Type declaration: %s = %s\n",
        name, type);

    count++;
}

```

```

void addVariable(char *name, char *type)
{
    strcpy(table[count].name, name);
    strcpy(table[count].type, type);
    table[count].isAlias = 0;
}

```

```

    count++;
}

char* lookupType(char *name)
{
    int i;

    for(i = 0; i < count; i++)
    {
        if(strcmp(table[i].name, name) == 0)
        {
            return table[i].type;
        }
    }

    return NULL;
}

int nameEquivalent(char *a, char *b)
{
    return strcmp(a, b) == 0;
}

int structuralEquivalent(char *a, char *b)
{
    char *typeA = lookupType(a);
    char *typeB = lookupType(b);

    if(typeA == NULL || typeB == NULL)
        return 0;

    return strcmp(typeA, typeB) == 0;
}

int main()
{
    printf("Enter type declarations:\n");

    yyparse();

    printf("\n=== Type Equivalence Test ===\n");

    if(nameEquivalent("A", "B"))
        printf("Name Equivalence: EQUIVALENT\n");
    else
        printf("Name Equivalence: NOT EQUIVALENT\n");

    if(structuralEquivalent("A", "B"))
        printf("Structural Equivalence: EQUIVALENT\n");
    else
        printf("Structural Equivalence: NOT EQUIVALENT\n");

    return 0;
}

```

```
void yyerror(const char *s)
{
    printf("Syntax Error: %s\n", s);
}
```

Step 3: Compile

```
flex q4.l
bison -d q4.y
gcc lex.yy.c q4.tab.c -o q4.exe
q4.exe
```

Step 4: Input

```
type A = int;
type B = int;
A x;
B y;
```

Step 5: Output

```
Type declaration: A = int
Type declaration: B = int
Variable x declared as type A
Variable y declared as type B
```

=== Type Equivalence Test ===

```
Name Equivalence: NOT EQUIVALENT
Structural Equivalence: EQUIVALENT
```

Q4 Explanation

Name Equivalence

Compare the names:

A

B

Since:

A != B

Result:

NOT EQUIVALENT

Structural Equivalence

Compare the underlying types:

A → int

B → int

Therefore:

int == int

Result:

EQUIVALENT

Final Table

Type Underlying Type

A int

B int

x A

y B

Equivalence

Result

Name Equivalence ✗ Not Equivalent

Equivalence	Result
Structural Equivalence	✔ Equivalent

Q5. Type Coercion – $\text{int} \rightarrow \text{float}$

Given:

```
float temperature;
```

```
int sensor_value;
```

```
temperature = sensor_value;
```

The compiler should automatically convert:

$\text{int} \rightarrow \text{float}$

Conceptually:

```
temperature = (float)sensor_value;
```

Step 1: Create q5.l

```
%{
#include "q5.tab.h"
#include <string.h>
#include <stdlib.h>
}%

%%

"float"    return FLOAT;
"int"      return INT;

[A-Za-z_][A-Za-z0-9_]* {
    yylval.str = strdup(yytext);
    return ID;
}

"="        return '=';

";"        return ';';

[ \t\n]+   ;

.          return yytext[0];

%%

int yywrap()
{
    return 1;
}
```

Step 2: Create q5.y

```
%{
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

int yylex();
void yyerror(const char *s);
```

```

typedef struct
{
    char name[30];
    char type[10];
} Symbol;

Symbol table[100];
int count = 0;

void addSymbol(char *name, char *type);
char* lookup(char *name);
%}

%union
{
    char *str;
}

%token INT FLOAT
%token <str> ID

%%

program:
    declarations assignments
    ;

declarations:
    declarations declaration
    | declaration
    ;

declaration:
    FLOAT ID ';'
    {
        addSymbol($2, "float");

        printf("Declared: %s as float\n", $2);
    }
    | INT ID ';'
    {
        addSymbol($2, "int");

        printf("Declared: %s as int\n", $2);
    }
    ;

assignments:
    assignments assignment
    | assignment
    ;

assignment:
    ID '=' ID ';'

```

```

{
    char *lhsType = lookup($1);
    char *rhsType = lookup($3);

    printf("\nAssignment Analysis:\n");

    printf("Left side : %s (%s)\n",
        $1, lhsType);

    printf("Right side: %s (%s)\n",
        $3, rhsType);

    if(strcmp(lhsType, "float") == 0 &&
        strcmp(rhsType, "int") == 0)
    {
        printf("Type Conversion: int -> float\n");

        printf("Implicit coercion performed.\n");

        printf("Equivalent operation: %s = (float)%s\n",
            $1, $3);
    }
    else if(strcmp(lhsType, rhsType) == 0)
    {
        printf("Types are compatible. No conversion required.\n");
    }
    else
    {
        printf("Type Error: Incompatible types.\n");
    }
}
;

%%

void addSymbol(char *name, char *type)
{
    strcpy(table[count].name, name);
    strcpy(table[count].type, type);

    count++;
}

char* lookup(char *name)
{
    int i;

    for(i = 0; i < count; i++)
    {
        if(strcmp(table[i].name, name) == 0)
        {
            return table[i].type;
        }
    }
}

```

```

    return "unknown";
}

void yyerror(const char *s)
{
    printf("Syntax Error: %s\n", s);
}

int main()
{
    printf("Enter program:\n");

    yyparse();

    printf("\n=== Symbol Table ===\n");

    for(int i = 0; i < count; i++)
    {
        printf("%s : %s\n",
            table[i].name,
            table[i].type);
    }

    return 0;
}

```

Step 3: Compile

```

flex q5.l
bison -d q5.y
gcc lex.yy.c q5.tab.c -o q5.exe
q5.exe

```

Step 4: Input

```

float temperature;
int sensor_value;
temperature = sensor_value;

```

Step 5: Output

```

Declared: temperature as float
Declared: sensor_value as int

```

Assignment Analysis:

Left side : temperature (float)

Right side: sensor_value (int)

Type Conversion: int -> float

Implicit coercion performed.

Equivalent operation: temperature = (float)sensor_value

=== Symbol Table ===

temperature : float

sensor_value : int

Q5 Explanation

Initially:

temperature → float

sensor_value → int

Assignment:
temperature = sensor_value;
The compiler sees:
float = int
It performs implicit conversion:
int → float
So internally it becomes:
temperature = (float)sensor_value;
This process is called:
Implicit Type Conversion / Type Coercion

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding; unable to integrate concepts
Experimental Design	30	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation
Use of Tools	20	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools
Analysis of Results	15	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results
Documentation	10	Clear, well-structured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation